

Módulo de Consultas Analíticas (SQL)

Estructura, optimización y desglose paso a paso para el sistema de gestión de salud

CONSULTA 1: Afiliados con Facturas Pendientes

Permite identificar a los usuarios afiliados que tienen deudas vigentes en el sistema y calcular el monto total acumulado pendiente de pago.

```
SELECT
    a.id_afiliado,
    u.nombre,
    u.email,
    COUNT(f.id_factura) AS facturas_pendientes,
    SUM(f.total) AS total_pendiente
FROM afiliados a
JOIN usuarios u ON a.id_usuario = u.id_usuario
LEFT JOIN facturas f ON a.id_afiliado = f.id_afiliado AND f.estado = 'PENDIENTE'
GROUP BY a.id_afiliado, u.nombre, u.email
HAVING COUNT(f.id_factura) > 0;
```

Explicación paso a paso:

- **Paso 1 (FROM):** Comenzamos la lectura de datos desde la tabla base de afiliados `a`.
- **Paso 2 (JOIN):** Cruzamos con la tabla usuarios `u` para obtener datos personales (nombre y email).
- **Paso 3 (LEFT JOIN):** Vinculamos con facturas `f` que cumplan estrictamente la condición de estar como 'PENDIENTE'.
- **Paso 4 (GROUP BY):** Agrupamos la información por la clave primaria del afiliado, su nombre y su correo electrónico.
- **Paso 5 (COUNT):** Contamos de manera exacta cuántas facturas pendientes tiene acumuladas cada registro.
- **Paso 6 (SUM):** Realizamos la sumatoria financiera del dinero total de la deuda.
- **Paso 7 (HAVING):** Filtramos los resultados post-agrupación para excluir a los afiliados que no registren deudas.

CONSULTA 2: Facturación por Plan de Salud

Muestra el nivel de ingresos percibido por cada uno de los planes de salud disponibles junto con el volumen total de afiliados únicos inscritos.

```
SELECT
    p.nombre,
    COUNT(DISTINCT a.id_afiliado) AS total_afiliados,
    COALESCE(SUM(f.total), 0) AS facturacion_total
FROM planes p
LEFT JOIN afiliados a ON p.id_plan = a.id_plan
LEFT JOIN facturas f ON a.id_afiliado = f.id_afiliado AND f.estado = 'PAGADA'
GROUP BY p.id_plan, p.nombre;
```

Explicación paso a paso:

- **Paso 1 (FROM):** Partimos desde la tabla de catálogo comercial de planes `p` de salud.
- **Paso 2 (LEFT JOIN):** Acoplamos la tabla de afiliados `a` correspondientes a cada plan de manera no exclusiva.
- **Paso 3 (LEFT JOIN):** Cruzamos con las facturas `f` asociadas que posean un estado igual a 'PAGADA'.
- **Paso 4 (GROUP BY):** Segmentamos y consolidamos las estadísticas usando el identificador y nombre del plan.
- **Paso 5 (COUNT DISTINCT):** Contamos los afiliados únicos por plan, previniendo duplicados por re-inscripción.
- **Paso 6 (COALESCE):** Sumamos los montos; si el plan no cuenta con ingresos, convierte el valor nulo (NULL) a 0.

CONSULTA 3: Diagnósticos Frecuentes por Especialidad

Ayuda a identificar cuáles son las patologías o motivos de consulta médica más recurrentes agrupados por rama de especialidad.

```
SELECT
    p.especialidad,
    h.diagnostico,
    COUNT(*) AS cantidad
FROM historial_clinico h
JOIN profesionales p ON h.id_profesional = p.id_profesional
GROUP BY p.especialidad, h.diagnostico
ORDER BY cantidad DESC
LIMIT 20;
```

Explicación paso a paso:

- **Paso 1 (FROM):** Iniciamos directamente desde la tabla masiva de anotaciones de `historial_clinico` `h`.
- **Paso 2 (JOIN):** Conectamos con `profesionales` `p` para asociar la consulta médica con la especialidad del doctor.
- **Paso 3 (GROUP BY):** Agrupamos bajo la tupla combinada de especialidad médica y descripción de diagnóstico.
- **Paso 4 (COUNT):** Contamos las ocurrencias totales de cada enfermedad registrada en el sistema.
- **Paso 5 (ORDER BY DESC):** Clasificamos el listado de mayor a menor frecuencia de casos médicos atendidos.
- **Paso 6 (LIMIT):** Truncamos el set de resultados finales para conservar únicamente las 20 principales filas.

CONSULTA 4: Centros Más Utilizados

Analiza de forma cuantitativa la afluencia y carga transaccional de pacientes sobre los diferentes centros de atención de la red de salud.

```

SELECT
    c.nombre,
    c.ciudad,
    COUNT(cit.id_cita) AS total_citas
FROM centros_salud c
LEFT JOIN citas cit ON c.id_centro = cit.id_centro
GROUP BY c.id_centro, c.nombre, c.ciudad
ORDER BY total_citas DESC;

```

Explicación paso a paso:

- **Paso 1 (FROM):** Tomamos el directorio oficial de sucursales en la tabla `centros_salud c`.
- **Paso 2 (LEFT JOIN):** Vinculamos el registro completo de la agenda de citas `cit` concertadas en cada sede física.
- **Paso 3 (GROUP BY):** Agrupamos los datos basándonos en el identificador único, nombre y ciudad de localización del centro.
- **Paso 4 (COUNT):** Contamos numéricamente las citas totales ligadas a cada locación física.
- **Paso 5 (ORDER BY DESC):** Ordenamos la lista priorizando los centros de salud que presentan mayor volumen operativo.

CONSULTA 5: Citas Próximas de un Afiliado

Extrae la agenda médica activa y cronológicamente más inmediata correspondiente a un paciente específico.

```

SELECT c.id_cita, c.fecha, c.estado,
       u.nombre AS profesional,
       p.especialidad,
       cs.nombre AS centro
FROM citas c
JOIN profesionales p ON c.id_profesional = p.id_profesional
JOIN usuarios u ON p.id_usuario = u.id_usuario
JOIN centros_salud cs ON c.id_centro = cs.id_centro
WHERE c.id_afiliado = ?
      AND c.fecha >= NOW()
      AND c.estado IN ('PENDIENTE', 'CONFIRMADA')
ORDER BY c.fecha
LIMIT 10;

```

Explicación paso a paso:

- **Paso 1 (FROM):** Consultamos la base operativa de datos de citas `c`.
- **Paso 2 y 3 (JOIN):** Enlazamos con profesionales `p` y posteriormente con usuarios `u` para obtener el nombre del médico.
- **Paso 4 (JOIN):** Integramos la tabla `centros_salud cs` para dar la ubicación física exacta del turno.
- **Paso 5 (WHERE):** Filtramos por el marcador de posición parametrizado (?) correspondiente al ID de un afiliado.
- **Paso 6 y 7 (AND):** Acotamos el resultado a fechas futuras (`NOW()`) y estados válidos ('PENDIENTE' o 'CONFIRMADA').

- **Paso 8 y 9 (ORDER / LIMIT):** Ordenamos la agenda de forma ascendente y retenemos los 10 registros más próximos.

CONSULTA 6: Profesionales por Centro de Salud

Consolida un directorio estructurado del personal médico en actividad clasificado según su sede o centro asignado.

```
SELECT
    cs.nombre AS centro,
    cs.ciudad,
    COUNT(p.id_profesional) AS total_profesionales,
    GROUP_CONCAT(u.nombre SEPARATOR ', ') AS nombres_profesionales
FROM centros_salud cs
LEFT JOIN profesionales p ON cs.id_centro = p.id_centro
LEFT JOIN usuarios u ON p.id_usuario = u.id_usuario
GROUP BY cs.id_centro, cs.nombre, cs.ciudad
ORDER BY total_profesionales DESC;
```

Explicación paso a paso:

- **Paso 1, 2 y 3 (FROM / JOINS):** Unimos centros de salud, profesionales y sus respectivos nombres de usuario.
- **Paso 4 (GROUP BY):** Agrupamos la información por el centro de salud y su metadata geográfica.
- **Paso 5 (COUNT):** Realiza el conteo global de médicos asignados por cada sucursal de la red.
- **Paso 6 (GROUP_CONCAT):** Agrupa horizontalmente y concatena en una sola celda los nombres completos separados por comas.

CONSULTA 7: Profesionales con Más Citas

Evalúa el volumen de trabajo histórico del personal médico desglosando la efectividad de sus citas asignadas.

```
SELECT
    u.nombre AS profesional,
    p.especialidad,
    cs.nombre AS centro,
    COUNT(c.id_cita) AS total_citas,
    SUM(CASE WHEN c.estado = 'FINALIZADA' THEN 1 ELSE 0 END) AS citas_finalizadas,
    SUM(CASE WHEN c.estado = 'CANCELADA' THEN 1 ELSE 0 END) AS citas_canceladas
FROM profesionales p
JOIN usuarios u ON p.id_usuario = u.id_usuario
LEFT JOIN centros_salud cs ON p.id_centro = cs.id_centro
LEFT JOIN citas c ON p.id_profesional = c.id_profesional
GROUP BY p.id_profesional, u.nombre, p.especialidad, cs.nombre
ORDER BY total_citas DESC;
```

Explicación paso a paso:

- **Paso 1 al 4:** Cruzamos profesionales, perfiles de usuario, centros médicos y el maestro histórico de citas médicas.

- **Paso 5 (GROUP BY):** Agrupamos con base en las propiedades de identificación del profesional de la salud.
- **Paso 6 (COUNT):** Extrae el volumen neto e histórico de interacciones agendadas.
- **Paso 7 y 8 (SUM CASE):** Sumadores lógicos condicionales que evalúan el estado e incrementan en uno si la cita concluyó o se canceló.

CONSULTA 8: Disponibilidad Semanal de un Profesional

Presenta de manera ordenada y legible el esquema de turnos semanales de atención configurado por los profesionales médicos.

```
SELECT
    u.nombre AS profesional,
    p.especialidad,
    d.dia_semana,
    CASE d.dia_semana
        WHEN 1 THEN 'Lunes'
        WHEN 2 THEN 'Martes'
        WHEN 3 THEN 'Miércoles'
        WHEN 4 THEN 'Jueves'
        WHEN 5 THEN 'Viernes'
        WHEN 6 THEN 'Sábado'
        WHEN 7 THEN 'Domingo'
    END AS dia_nombre,
    d.hora_inicio,
    d.hora_fin
FROM disponibilidad_profesional d
JOIN profesionales p ON d.id_profesional = p.id_profesional
JOIN usuarios u ON p.id_usuario = u.id_usuario
WHERE d.activo = TRUE
ORDER BY u.nombre, d.dia_semana, d.hora_inicio;
```

Explicación paso a paso:

- **Paso 1, 2 y 3:** Consultamos la agenda de disponibilidad cruzándola con el profesional y su nombre.
- **Paso 4 (WHERE):** Filtramos estrictamente los horarios marcados en el sistema como activos (TRUE).
- **Paso 5 (CASE):** Estructura de mapeo lógico que traduce los valores numéricos estándar (1 al 7) a nombres de días.
- **Paso 6 (ORDER BY):** Ordenamiento por el nombre del profesional, seguido del orden correlativo de los días y horas de inicio.

CONSULTA 9: Profesionales por Especialidad

Entrega un censo demográfico de la cantidad de talento corporativo operando según su correspondiente especialización médica.

```
SELECT
    p.especialidad,
    COUNT(p.id_profesional) AS total_profesionales,
    GROUP_CONCAT(u.nombre SEPARATOR ', ') AS nombres
FROM profesionales p
JOIN usuarios u ON p.id_usuario = u.id_usuario
GROUP BY p.especialidad
ORDER BY total_profesionales DESC;
```

Explicación paso a paso:

- **Paso 1 y 2:** Conectamos la tabla de médicos con los datos de sus respectivas cuentas en usuarios.
- **Paso 3 (GROUP BY):** Concentramos el análisis agrupando por la columna de especialidad médica.
- **Paso 4 y 5 (COUNT / CONCAT):** Suma las cabezas médicas por rama y lista secuencialmente sus nombres en formato de texto.

CONSULTA 10: Citas por Estado

Reporta indicadores operativos de salud a través de volúmenes absolutos y participaciones porcentuales de las citas médicas.

```
SELECT
    c.estado,
    COUNT(c.id_cita) AS total,
    ROUND(COUNT(c.id_cita) * 100.0 / (SELECT COUNT(*) FROM citas), 2) AS porcentaje
FROM citas c
GROUP BY c.estado
ORDER BY total DESC;
```

Explicación paso a paso:

- **Paso 1 y 2:** Leemos de la tabla de citas y segmentamos agrupando por el campo descriptivo del estado del turno.
- **Paso 3 (COUNT):** Cuantifica de forma directa el número neto de registros presentes en cada estado posible.
- **Paso 4 (ROUND):** Calcula el porcentaje dividiendo el subtotal entre el gran total de una subconsulta, fijando dos decimales.

CONSULTA 11: Distribución de Citas por Día de la Semana

Identifica los días calendario laborables que concentran la mayor afluencia de pacientes a la red asistencial.

```

SELECT
    CASE DAYOFWEEK(c.fecha)
        WHEN 1 THEN 'Domingo'
        WHEN 2 THEN 'Lunes'
        WHEN 3 THEN 'Martes'
        WHEN 4 THEN 'Miércoles'
        WHEN 5 THEN 'Jueves'
        WHEN 6 THEN 'Viernes'
        WHEN 7 THEN 'Sábado'
    END AS dia_semana,
    COUNT(c.id_cita) AS total_citas
FROM citas c
GROUP BY DAYOFWEEK(c.fecha)
ORDER BY FIELD(DAYOFWEEK(c.fecha), 2, 3, 4, 5, 6, 7, 1);

```

Explicación paso a paso:

- **Paso 1 y 2:** Empleamos la función DAYOFWEEK () para extraer el valor de índice semanal de la columna de fecha de cita.
- **Paso 3 y 4:** Convertimos el índice numérico a texto legible y agrupamos bajo este criterio cronológico.
- **Paso 6 (ORDER BY FIELD):** Fuerza de forma artificial que los renglones se presenten en orden secuencial de Lunes a Domingo.

CONSULTA 12: Horarios Pico de Citas

Determina las franjas horarias con mayor congestión en la asignación y atención de turnos médicos.

```

SELECT
    HOUR(c.fecha) AS hora,
    COUNT(c.id_cita) AS total_citas
FROM citas c
GROUP BY HOUR(c.fecha)
ORDER BY total_citas DESC;

```

Explicación paso a paso:

- **Paso 2 (HOUR):** Ejecuta la función nativa que aísla el componente de la hora (formato militar de 0 a 23).
- **Paso 3 y 4:** Agrupa por bloques horarios de 60 minutos y computa la cantidad total de citas cargadas en el sistema.

CONSULTA 13: Ingresos Mensuales

Proporciona una vista contable histórica de los ingresos recaudados agrupados de manera mensual.

```
SELECT
    YEAR(p.fecha) AS anio,
    MONTH(p.fecha) AS mes,
    COUNT(p.id_pago) AS total_pagos,
    SUM(p.monto) AS ingresos_totales
FROM pagos p
GROUP BY YEAR(p.fecha), MONTH(p.fecha)
ORDER BY anio DESC, mes DESC;
```

Explicación paso a paso:

- **Paso 2, 3 y 4:** Descomponemos el atributo temporal de pagos en campos numéricos independientes de año y mes para agruparlos.
- **Paso 5 y 6 (COUNT / SUM):** Consolida la cantidad neta de operaciones de pago procesadas junto con el dinero líquido total capturado.

CONSULTA 14: Ingresos por Centro de Salud

Establece el aporte financiero que cada locación física genera para la organización médica.

```
SELECT
    cs.nombre AS centro,
    cs.ciudad,
    COUNT(p.id_pago) AS total_pagos,
    COALESCE(SUM(p.monto), 0) AS ingresos_totales
FROM centros_salud cs
LEFT JOIN citas c ON cs.id_centro = c.id_centro
LEFT JOIN facturas f ON c.id_afiliado = f.id_afiliado AND f.estado = 'PAGADA'
LEFT JOIN pagos p ON f.id_factura = p.id_factura
GROUP BY cs.id_centro, cs.nombre, cs.ciudad
ORDER BY ingresos_totales DESC;
```

Explicación paso a paso:

- **Paso 1 al 4 (JOINS):** Une centros de salud con citas, mapeando con facturas solventadas y sus cobros definitivos.
- **Paso 7 (COALESCE):** Protege la consulta financiera inyectando un valor de cero si la sede no reportó transacciones comerciales.

CONSULTA 15: Distribución por Método de Pago

Estudia la preferencia en el uso de pasarelas o medios de pago financieros por parte de los afiliados.


```

SELECT
    p.metodo_pago,
    COUNT(p.id_pago) AS total_transacciones,
    SUM(p.monto) AS monto_total,
    ROUND(SUM(p.monto) * 100.0 / (SELECT SUM(monto) FROM pagos), 2) AS porcentaje
FROM pagos p
GROUP BY p.metodo_pago
ORDER BY monto_total DESC;

```

CONSULTA 16: Distribución de Afiliados por Plan

Entrega métricas comerciales de la penetración del catálogo de planes de cobertura médica sobre el total de la cartera de clientes.

```

SELECT
    pl.nombre AS plan,
    pl.costo,
    COUNT(a.id_afiliado) AS total_afiliados,
    ROUND(COUNT(a.id_afiliado) * 100.0 / (SELECT COUNT(*) FROM afiliados), 2) AS
porcentaje
FROM planes pl
LEFT JOIN afiliados a ON pl.id_plan = a.id_plan
GROUP BY pl.id_plan, pl.nombre, pl.costo
ORDER BY total_afiliados DESC;

```

CONSULTA 17: Afiliados Nuevos por Mes

Establece indicadores clave de rendimiento (KPI) relativos al crecimiento mensual de nuevos usuarios.

```

SELECT
    YEAR(u.fecha_creacion) AS anio,
    MONTH(u.fecha_creacion) AS mes,
    COUNT(u.id_usuario) AS nuevos_afiliados
FROM usuarios u
WHERE u.rol = 'AFILIADO'
GROUP BY YEAR(u.fecha_creacion), MONTH(u.fecha_creacion)
ORDER BY anio DESC, mes DESC;

```

Resumen de Funciones SQL Usadas

A continuación se consolidan los comandos y funciones utilizados a lo largo del módulo analítico:

Función / Operador	¿Para qué sirve?	Consultas Aplicadas
JOIN	Une registros vinculando tablas mediante llaves primarias y foráneas exactas.	Todas las consultas
LEFT JOIN	Une tablas preservando filas de la tabla izquierda aunque no existan en la derecha.	1, 2, 4, 6, 14, 16
GROUP BY	Agrupar filas con valores idénticos para realizar funciones de agregación.	Todas las consultas
COUNT() / SUM()	Computa el recuento total de filas o calcula la sumatoria aritmética de montos.	Todas las consultas
COALESCE()	Evalúa los argumentos en orden y retorna el primer elemento no nulo.	2, 14
CASE WHEN	Aplica lógica condicional secuencial inline tipo IF-THEN-ELSE dentro del SELECT.	7, 8, 11
GROUP_CONCAT()	Agrupar textos procedentes de varias filas en una sola celda usando un delimitador.	6, 9
HAVING	Efectúa un filtro estricto sobre datos agregados (operación post-GROUP BY).	1
LIMIT	Acota la cantidad máxima de filas devueltas en el set de datos final.	3, 5
YEAR() / MONTH() / HOUR()	Funciones cronológicas que extraen partes atómicas de tipos de datos DATETIME.	11, 12, 13, 17
ROUND()	Redondea un número flotante al número especificado de decimales.	10, 15, 16
FIELD()	Permite forzar un orden de ordenamiento según una lista personalizada fija.	11